Contenido

- Introducción
- Funciones Nativas
- Funciones Definidas
- Funciones Anónimas
- Funciones de flecha (Arrow functions)
- Funciones Anónimas vs Arrow functions
- Ámbito de Variables
- Variables Static

Programación Estructurada en PHP

La programación estructurada en PHP 8.3 es un paradigma que organiza el código en bloques lógicos y secuenciales, utilizando principalmente funciones y estructuras de control como `if`, `for` y `while`. Aunque PHP es un lenguaje que soporta múltiples paradigmas, la programación estructurada constituye su base y es fundamental para escribir código claro, eficiente y fácil de mantener, especialmente en proyectos de pequeña a mediana escala.

Principios Fundamentales

El paradigma se basa en el teorema del programa estructurado, que establece que cualquier programa puede ser escrito utilizando únicamente tres tipos de estructuras de control:

- **Secuencia:** Las instrucciones del código se ejecutan en orden, una después de la otra, de arriba hacia abajo.
- **Selección (o Condicional):** Permite que el programa tome decisiones y ejecute diferentes bloques de código según se cumpla o no una condición. En PHP, esto se implementa con las estructuras `if-else` y `switch`.
- **Iteración (o Bucle):** Permite repetir un bloque de código varias veces mientras se cumpla una condición. Las estructuras utilizadas en PHP son los bucles `for`, `while`, y `do-while`.

Un pilar adicional es la **Modularización**: el programa se divide en módulos o bloques más pequeños, conocidos como **funciones**, que realizan tareas específicas. Esto fomenta la reutilización del código y simplifica la lógica general.

Ejemplo en la Práctica

Un script estructurado en PHP para verificar números pares podría verse así:

```php
<?php

// Definición de una función (Modularización)
function esPar($numero) {
    // Uso de selección para tomar una decisión
    if ($numero % 2 === 0) {
        return true;
    } else {
        return false;
    }
}

// Bucle que recorre números del 1 al 10 (Iteración)
for ($i = 1; $i <= 10; $i++) {
    // El código se ejecuta en secuencia
    echo "El número $i ";
    if (esPar($i)) {
        echo "es par ";
    } else {
        echo "es impar";
    }
}

?>
```

Relevancia en PHP Moderno

Aunque la programación estructurada es perfectamente funcional en PHP 8.3, en el desarrollo de aplicaciones complejas, el paradigma dominante es la **Programación Orientada a Objetos (POO)**. La POO ofrece un nivel superior de organización al agrupar datos y funciones en "objetos".

Comparativa: Programación Estructurada vs. POO

CARACTERÍSTICA	PROGRAMACIÓN ESTRUCTURADA	PROGRAMACIÓN ORIENTADA A OBJETOS (POO)
Enfoque	En procedimientos y funciones lógicas.	En entidades (objetos) que tienen datos y comportamiento.
Unidad principal	Funciones y estructuras de control.	Clases y objetos.
Reusabilidad	A través de funciones que se pueden llamar.	A través de herencia y polimorfismo.
Ideal para	Scripts, utilidades, y aplicaciones pequeñas.	Aplicaciones complejas, grandes sistemas y frameworks.

Funciones Nativas

Funciones de Strings

strlen
Obtiene la longitud de una cadena, es decir, el número de caracteres.

```php
<?php
  $cad = "cadena";
  $n = strlen($cad);
  echo "La longitud de la cadena es: $n";
?>
```

substr
Devuelve una subcadena, tomando:

- Cadena original
- Posición inicial
- Cantidad de caracteres (opcional)

```php
<?php
  $cad = "cadena";
  $sub1 = substr($cad, 2);
  $sub2 = substr($cad, 2, 2);
  echo "Subcadena 1: $sub1 \n";
  echo "Subcadena 2: $sub2";
?>
```

strtoupper
Convierte un string a mayúsculas.

```php
<?php
$str = "Mi mamá me mima";
$str = strtoupper($str);
echo $str; // MI MAMÁ ME MIMA
?>
```

Funciones de Manejo de Variables

is_array - Comprueba si un valor es un array
is_bool - Comprueba si es boolean
isset - Verifica que la variable está definida y no es NULL
print_r - Imprime información legible
var_dump - Detalla tipo y valor de variable

Funciones Definidas por el Usuario

Paso de Parámetros y Retorno

```php
<?php
function cuadrado($num) {
  return $num * $num;
}
echo cuadrado(4); // 16
?>
```

Variables por Referencia y Valor

Valor: copia el valor en nueva zona. Modificaciones no afectan la variable original.
Referencia: variable nueva apunta a la misma zona de memoria que la original. Modificaciones afectan a ambas.

```php
<?php
  $a = 5;
  $b =& $a; // referencia
  $c = $a;  // valor
  $b = 6;
  echo "$a, $b, $c"; // 6, 6, 5
?>
```

Uso de Parámetros Predeterminados

```php
<?php
  function hacer_cafe($tipo = "capuchino") {
    return "Preparando una taza de $tipo.";
  }
  echo hacer_cafe();        // capuchino
  echo hacer_cafe("espresso"); // espresso
?>
```

Funciones Anónimas en PHP

Las funciones anónimas, también conocidas como *closures*, son funciones que no tienen nombre y se pueden asignar a variables o pasar como parámetros a otras funciones. Son muy útiles para realizar tareas específicas sin necesidad de declararlas con un nombre.

Ejemplo básico de función anónima

```php
<?php
$saludar = function($nombre) {
    return "Hola, $nombre";
};

echo $saludar("Mundo"); // Salida: Hola, Mundo
?>
```

Funciones anónimas con variables externas (uso de `use`)

```php
$mensaje = "Buenos días";

$saludar = function($nombre) use ($mensaje) {
    return "$mensaje, $nombre";
};

echo $saludar("Ana"); // Salida: Buenos días, Ana
?>
```

Este recurso te permite mantener el código modular, compacto y claro, especialmente cuando se usa para callbacks o tareas puntuales.

Arrow Functions (Funciones Flecha)

Introducidas en **PHP 7.4**, las funciones flecha son una sintaxis más corta y limpia para escribir funciones anónimas. Su principal objetivo es simplificar el código, especialmente cuando se usan como *callbacks* en funciones de arrays como `array_map` o `array_filter`.

Características Principales

- **Sintaxis concisa:** Se definen con la palabra clave `fn`, seguida de los parámetros y el operador `=>`.
- **Retorno implícito:** No necesitan la palabra clave `return`. La expresión que sigue a `=>` es lo que la función devuelve automáticamente.
- **Captura automática del ámbito:** Acceden a las variables del ámbito padre (scope) de forma automática, sin necesidad de usar la cláusula `use`.
- **Limitación a una expresión:** Solo pueden contener una única expresión. No se pueden escribir múltiples líneas de código o declaraciones complejas.

Comparativa: Función Anónima vs. Arrow Function

La diferencia es muy visual. Veamos el mismo ejemplo de las dos formas.

1. Sin variables externas

```php
<?php
$numeros = [1, 2, 3, 4];

// Forma tradicional con función anónima
$dobles_tradicional = array_map(function($n) {
    return $n * 2;
}, $numeros);

// Forma moderna con Arrow Function
$dobles_flecha = array_map(fn($n) => $n * 2, $numeros);
?>
```

2. Con variables externas

Aquí es donde brilla la simplicidad de las funciones flecha, ya que no requieren la cláusula `use`.

```php
<?php
$multiplicador = 10;
$numeros = [1, 2, 3, 4];

// Forma tradicional (necesita 'use' para acceder a $multiplicador)
$resultado_tradicional = array_map(function($n) use ($multiplicador) {
    return $n * $multiplicador;
}, $numeros);

// Con Arrow Function (accede a $multiplicador automáticamente)
$resultado_flecha = array_map(fn($n) => $n * $multiplicador, $numeros);
?>
```

💡 **Consejo del profesor:**
Usa una **Arrow Function** siempre que tu lógica se pueda resolver en una única expresión. Para funciones más complejas que requieran múltiples líneas, condiciones o bucles, sigue usando una **función anónima tradicional** con sus llaves `{}`.

Explicación: ¿Qué es la "Captura Automática del Ámbito"?

"Capturar el ámbito" significa que una función puede "ver" y utilizar las variables que existen fuera de ella, en el mismo lugar donde fue creada. La diferencia clave entre las funciones anónimas tradicionales y las funciones flecha es cómo lo hacen:

1. Función Anónima (`function`)

Por defecto, es una "burbuja" aislada. No puede ver las variables externas. Para darle acceso, debemos pasarle un "permiso" explícito usando la cláusula **use**.

```php
<?php
$multiplicador = 10;

// Necesita 'use' para "heredar" $multiplicador
$funcion = function($n) use ($multiplicador) {
    return $n * $multiplicador;
};

echo $funcion(5); // Salida: 50
?>
```

2. Función Flecha (`fn`)

Es una "burbuja transparente". **Automáticamente** ve y utiliza las variables de su entorno, sin necesidad de pedir permiso con `use`.

```php
<?php
$multiplicador = 10;

// NO necesita 'use'. Accede a $multiplicador
// de forma automática.
$funcionFlecha = fn($n) => $n * $multiplicador;

echo $funcionFlecha(5); // Salida: 50
?>
```

💡 **Nota Importante sobre la Captura:**
En ambos casos, por defecto, la variable se captura **por valor**. Esto significa que la función utiliza una copia del valor que tenía la variable en el momento en que la función fue definida. Si el valor de la variable externa cambia después, la función no se enterará.

Una Analogía Final

Imagina que estás en tu despacho (función anónima) y necesitas un libro que está en la biblioteca (ámbito padre).

- **Con function y use:** Tienes que rellenar un formulario (use) pidiendo permiso para sacar ese libro específico. Si no lo pides, el bibliotecario no te lo da.
- **Con fn (arrow function):** Tienes un pase especial que te permite coger cualquier libro de la biblioteca directamente, sin pedir permiso cada vez.

💡 **Importante:**
Esta captura automática es por valor, no por referencia. Esto significa que la función flecha obtiene una copia del valor de la variable en el momento en que se define. Si intentas modificar la variable externa desde dentro de la función flecha, no afectará a la original.

CARACTERÍSTICA	FUNCIÓN ANÓNIMA (FUNCTION)	FUNCIÓN FLECHA (FN)
Sintaxis	Más verbosa: `function() {}` con `return` explícito.	Concisa: `fn() => ...` con retorno implícito.
Captura de Ámbito	Requiere permiso explícito con la cláusula `use`.	Automática. Accede a las variables del padre sin `use`.
Modificar Ámbito Externo	Sí, pasando variables por referencia con `use (&$var)`.	No. La captura es siempre por valor (una copia).
Complejidad del Cuerpo	Permite múltiples líneas, bucles y lógica compleja.	Limitada a **una sola expresión**.
Caso de Uso Ideal	Callbacks complejos o que necesiten modificar variables externas.	Callbacks simples y de una sola línea (ej: en `array_map`, `array_filter`).

Ámbito de las Variables

El ámbito es el contexto donde una variable está definida. Variables dentro de una función tienen ámbito local, fuera tienen alcance global. Variables globales se deben declarar explícitamente con palabra clave `global` o usando `$GLOBALS`.

```php
<?php
  $a = 1;
  function test() {
    global $a;
    echo $a;
  }
  test();
 ?>
```

Uso de Variables Static

Variables estáticas mantienen su valor entre llamadas a función.

```php
<?php
  function contador() {
    static $count = 0;
    $count++;
    echo $count;
  }
  contador(); // 1
  contador(); // 2
?>
```

Este documento se ha realizado por Marián Capilla Herrero. Publicada bajo licencia
Creative Commons Atribución/Reconocimiento-CompartirIgual 4.0 Internacional.
Las modificaciones son permitidas siempre que sean mencionadas, la licencia no se puede modificar